

Costruiamo RecipeBot.

Un assistente di cucina, da 15 righe a un agente che osserva, agisce, ragiona, consulta e ricorda.

-
- 01 Sei dimensioni di un agente moderno, una alla volta
 - 02 Tutto in Python, in livecoding, con un singolo `recipebot.py` che cresce
 - 03 Alla fine: lo stesso agente, in scala, con l'aggancio alla Lezione 11

Cosa esploreremo oggi

Sei dimensioni che ogni agente moderno deve coprire.

01	
Context	cosa vede il modello a ogni run
02	
Tools	l'agente agisce nel mondo
03	
Reasoning	l'agente ragiona prima/durante le azioni
04	
Knowledge	consulta fonti esterne
05	
Storage / Sessions	ricorda la conversazione
06	
Memory	ricorda fatti sull'utente

Quando chiami `agent.run()`, cosa succede davvero?

Non una chiamata al modello — un loop. Lo stesso pattern ReAct che conoscete dall'SDK OpenAI, qui orchestrato dal framework.

1 Costruisce il context

`system + user + history + memory + state`

2 Chiama il modello

`request → LLM provider`

3 Il modello risponde con...

└ messaggio finale → ESCE

└ tool call → continua

4 Esegue il tool

aggiunge il risultato al context, torna a ②

Senza tool: una sola iterazione, esce subito.

Con tool: il modello decide quanti giri.

Il framework non sceglie per lui — orchestra il loop e lo rende osservabile.

AgentOS — il sistema operativo del vostro agente

Non solo un agente in uno script Python: un servizio osservabile, persistente, ispezionabile.

Runtime

`agno.os.AgentOS`

- Server FastAPI auto-generato
- 50+ endpoint REST + streaming SSE
- Sessioni e tracing nel database

Control Plane

`https://os.agno.com`

- UI web di gestione
- Connessione diretta dal browser
- Ispezione di sessioni, memorie, trace, knowledge

Il database è vostro. Sessioni, memorie, knowledge, trace — niente esce verso Agno.

La radiografia di una run

Per migliorare un agente serve prima vedere cosa fa.

Trace

Una run completa, dall'input dell'utente alla risposta finale.

Identificata da un `trace_id`.

Span

Una singola operazione dentro la trace (chiamata al modello, esecuzione di un tool, retrieval). Gli span hanno relazioni parent-child.

```
agent.run [2.3s]
├─ model.response [1.8s]
│   ├── tokens_in: 142
│   └── tokens_out: 87
└─ tool.execution: get_weather [0.4s]
    ├── args: {"city": "Milano"}
    └── result: "18°C, sereno"
```


STRESS TEST

Mettiamolo subito alla prova

Un prompt realistico, con vincoli espliciti. Cosa risponderà il nostro agente da 15 righe?

Voglio una cena vegetariana per due persone, massimo 30 minuti.

Ho ceci, zucchine, riso e yogurt greco.

Sono allergico alla frutta secca.

Non voglio comprare più di due ingredienti.

COSA OSSERVARE NELLA TRACE

- ☐ Ha rispettato tutti i vincoli?
- ☐ Ha distinto vincoli hard da preferenze soft?
- ☐ Ha inventato informazioni?
- ☐ Ha usato strumenti?
- ☐ Sappiamo perché ha scelto quella ricetta?

Plausibile non significa affidabile

L'agente ha risposto in modo convincente. Ma...

CATEGORIA	COSA FA L'AGENTE	COSA NON FA
Vincoli	Tiene conto di "vegetariano" nel testo	Non verifica che gli ingredienti siano davvero vegetariani
Tempi	Risponde "30 minuti"	Non ha modo di calcolarlo davvero
Strumenti	Niente	Non chiama nessun tool
Memoria	Niente	Non ricorda chi sei tra una sessione e l'altra
Conoscenza	Inventa ricette plausibili	Non consulta nessun ricettario reale

Le sei dimensioni della mappa-bussola sono le risposte a queste mancanze. Le aggiungiamo una alla volta nel resto della lezione.

Cosa entra nel prompt finale?

Lo stesso fatto può andare in posti diversi. La scelta influenza affidabilità, costo, debugging.

`instructions`

Regole di comportamento, valide per ogni run.

`history`

Cosa è stato detto finora nella sessione.

`session_state`

Stato strutturato della sessione (dict editabile).

`knowledge`

Contenuti esterni consultati on-demand.

`memory`

Fatti persistenti sull'utente, cross-sessione.

`tools`

Calcoli o dati ottenuti da codice deterministico.

`debug_mode=True` *mostra il prompt finale assemblato — la verità su cosa vede davvero il modello.*

Prima di scrivere un tool, classifica il rischio

Il decoratore `@tool` è facile. La domanda difficile è cosa succede se l'agente lo chiama nel modo sbagliato.

TIPO DI TOOL	ESEMPIO PER RECIPEBOT	RISCHIO	POLICY
Read-only	<code>get_available_ingredients()</code>	BASSO	libero
Computazionale	<code>estimate_missing_ingredients()</code>	BASSO	libero
Recupero dati esterni	<code>search_recipes_online()</code>	MEDIO	loggare la fonte
State update	<code>save_shopping_list()</code>	MEDIO	conferma utente
External action	<code>send_order_to_store()</code>	ALTO	HITL obbligatorio

HITL = Human-in-the-Loop. Lo vedremo nella Lezione 11.

Dove siamo

Tre dimensioni viste, tre da vedere.

01	cosa vede il modello a ogni run	FATTO
02	l'agente agisce nel mondo	FATTO
03	l'agente ragiona prima/durante le azioni	IN CORSO
04	consulta fonti esterne	
05	ricorda la conversazione	
06	ricorda fatti sull'utente	

Dove avviene il ragionamento?

Agno offre tre risposte. Dietro ognuna c'è un trade-off diverso.

	REASONING MODELS	REASONINGTOOLS	REASONING AGENTS
Dove	Interno al modello	Tool espliciti think(), analyze()	Prompt engineering strutturato
Osservabilità	Opaca o parziale	Ogni think() è uno span	Iterazioni visibili
Controllo	Zero (il modello pensa sempre)	Selettivo (l'agente decide)	Forzato (ragiona su ogni richiesta)
Costo	Incluso nel modello reasoning	Solo quando serve	Variabile (>1.5x)
Caso d'uso	Problema autocontenuto	ReAct con tool call	Garanzia sistematica

Per RecipeBot useremo **ReasoningTools**. È il caso prototipico: tool, vincoli da decomporre, alternative da valutare.

Chi decide quando cercare?

Stessa pipeline RAG sotto. Scelta architetturale completamente diversa sopra.

	RAG CLASSICA	AGENTIC RAG
Quando cerca	Sempre, su ogni query utente	Solo quando l'agente decide di farlo
Cosa cerca	La query letterale	Una query riformulata dall'agente
Quanti retrieval	Uno fisso	Zero, uno o più (anche iterativi)
Decisione	Hardcoded nel codice	Dell'agente, a runtime
Costo	Sempre fisso	Variabile, spesso più basso

Non è "agentic sempre meglio". Per query factuali stabili, la classica è prevedibile e ottima. L'agentic brilla quando le query sono complesse o ambigue.

In Agno entrambe coesistono: `add_knowledge_to_context=True` per la classica, `search_knowledge=True` per l'agentic.

Dove siamo

Quattro dimensioni viste, due da vedere — quelle sulla memoria.

01	cosa vede il modello a ogni run	FATTO
Context		
02	l'agente agisce nel mondo	FATTO
Tools		
03	l'agente ragiona prima/durante le azioni	FATTO
Reasoning		
04	consulta fonti esterne	FATTO
Knowledge		
05	ricorda la conversazione	IN CORSO
Storage / Sessions		
06	ricorda fatti sull'utente	
Memory		

DISTINZIONI

Tre confini sottili e fondamentali

Nel linguaggio comune sono tutti "memoria". Tecnicamente sono tre cose diverse.

	STORAGE / SESSIONS	MEMORY	KNOWLEDGE
Cosa contiene	Storia della conversazione	Fatti sull'utente	Contenuti del dominio
Scope	Per sessione	Per utente, cross-sessione	Globale (o filtrabile)
Esempio	"L'utente ha chiesto delle ricette vegetariane"	"L'utente è vegetariano"	"La cottura dei ceci richiede ammollo"
Domanda	Cosa abbiamo detto?	Cosa so di te?	Cosa so del dominio?

Dove metti "l'utente è vegetariano" determina la sua persistenza, il suo scope, e chi può vederlo.

Sei tipi di "memoria", non uno

Ogni store cattura un tipo diverso di informazione, con scope diverso.

STORE	COSA CATTURA	SCOPE
User Profile	Campi strutturati, schema fisso, aggiornati in place	Per utente
User Memory	Testo libero, osservazioni, appese nel tempo	Per utente
Session Context	Stato corrente della conversazione, sostituito a ogni update	Per sessione
Entity Memory	Fatti su entità esterne (companies, persons, projects)	Configurabile
Learned Knowledge	Insight trasferibili tra utenti	Configurabile
Decision Log	Decisioni con motivazione e outcome, per audit	Per agente

Profile vs Memory. *Profile = schema fisso, exact recall, aggiornato in place. Memory = testo libero, semantic search, appeso nel tempo. Nome o ruolo → Profile. Preferenze e osservazioni → Memory.*

Per RecipeBot: solo i primi due. Gli altri quattro tornano utili in scenari più complessi (assistenti professionali, sistemi audited, knowledge collaborative).

Quando l'agente impara?

Tre risposte, tre trade-off. Si possono mescolare per store diverso.

Always

estrazione automatica

Dopo ogni risposta, una chiamata LLM dedicata estrae fatti rilevanti.

PRO cattura garantita

CONTRO una chiamata in più per ogni run

Agentic

l'agente decide

Il modello principale riceve tool tipo `update_profile`, `update_user_memory`, e decide lui quando salvare.

PRO nessuna chiamata extra, scelte contestuali

CONTRO può perdere informazioni implicite

Propose

l'utente conferma

L'agente propone una memoria, ma chiede conferma esplicita.

PRO zero memorie indesiderate

CONTRO friction nell'esperienza utente

Per RecipeBot useremo **Agentic** per User Profile e User Memory. È più trasparente nelle trace.

RecipeBot v1 è completo

Sei dimensioni, un singolo `recipebot.py`. Un agente che osserva, agisce, ragiona, consulta e ricorda.

01			
Context	cosa vede il modello a ogni run		FATTO
02			
Tools	l'agente agisce nel mondo		FATTO
03			
Reasoning	l'agente ragiona prima/durante le azioni		FATTO
04			
Knowledge	consulta fonti esterne		FATTO
05			
Storage / Sessions	ricorda la conversazione		FATTO
06			
Memory	ricorda fatti sull'utente		FATTO

Funziona — ma come lo sappiamo?

"Sembra giusto" non è una metrica. La Lezione 11 inizia da qui.

| Quante volte rispetta davvero le allergie?

| Quante volte chiama `find_recipes_by_constraints` quando dovrebbe?

| Quante volte inventa una ricetta che non esiste nel ricettario?

| Quante volte le memorie estratte sono corrette?

| Senza una risposta numerica a queste domande, ogni modifica al prompt o al codice è una scommessa. **Evals** = trasformare l'intuizione in numeri.

Cosa vedremo nella Lezione 11

RecipeBot v2 — da agente che funziona ad agente governabile.

Evals

Misurare il comportamento, non descriverlo.

Workflow

Orchestrare passi multipli con guardie esplicite.

Team

Più agenti specializzati che collaborano.

HITL

L'umano nel loop sui tool ad alto rischio.

Stessa filosofia, scala diversa.